

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260619_105952_745295 -->

Framework: JUnit | Run ID: run_20260619_105952_745295

Tóm tắt kỹ thuật

- Framework: JUnit 5 (JUnit Jupiter)
- Kết quả chạy: Tất cả 13 test được thực thi, không có lỗi (Tests run: 13, Failures: 0, Errors: 0).
- Coverage: 96.27 % (đạt quality gate “coverage”).
- Loại vấn đề thực thi: Không có vấn đề (issue_type: none).
- Trình tự kiểm thử được đánh giá: BankAccountTest.java so với BankAccount.java.
- Trình tự kiểm thử còn thiếu: Các kịch bản liên quan tới @TempDir, migration JUnit 4 → JUnit 5 và @ParameterizedTest argument conversion không được hiện thực trong mã test.

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- @TempDir chia sẻ tĩnh giữa nhiều phương thức test
- Bằng chứng: Yêu cầu và validation trong Requirement/Validation đề cập tới “Test @TempDir shared static directory across multiple test methods”, nhưng trong BankAccountTest.java không có bất kỳ annotation @TempDir nào.
- Ảnh hưởng: Không kiểm chứng được hành vi JUnit 5 về thư mục tạm thời, gây thiếu sót trong phạm vi kiểm thử đã được yêu cầu.
- Cách sửa: Thêm một trường static @TempDir Path sharedTempDir; và viết ít nhất hai test method sử dụng chung thư mục này để xác nhận tính chia sẻ.
- Migration JUnit 4 → JUnit 5 (annotation thay thế)
- Bằng chứng: Các test case TC-014 và TC-015 trong Test Plan được đánh dấu *out_of_scope* vì không có mã nguồn liên quan; tuy nhiên yêu cầu vẫn yêu cầu “Kiểm thử việc sử dụng @TempDir...”, “Kiểm thử yêu cầu khi di chuyển từ JUnit 4 sang JUnit 5”.
- Ảnh hưởng: Không có test nào xác nhận rằng các annotation JUnit 4 (@Category, @RunWith, @Rule, @ClassRule) được thay bằng @Tag, @ExtendWith, @RegisterExtension.
- Cách sửa: Tạo một lớp test mẫu (có thể là dummy) sử dụng các annotation JUnit 4 cũ, sau đó viết test JUnit 5 để kiểm tra rằng chúng được thay thế đúng (ví dụ: kiểm tra sự tồn tại của @Tag trên lớp mới).
- @ParameterizedTest argument conversion (factory method vs constructor)
- Bằng chứng: Không có test nào sử dụng @ParameterizedTest, @ValueSource hay khai báo factory method/constructor trong dự án.
- Ảnh hưởng: Không thỏa mãn validation “factory method được ưu tiên hơn factory constructor khi cả hai cùng tồn tại” và “nhiều factory methods bị bỏ qua”.
- Cách sửa: Thêm một class ví dụ (ví dụ Currency) với một factory method public static Currency of(String s) và một constructor public Currency(String s). Viết test @ParameterizedTest để xác nhận rằng JUnit chọn factory method.

Assertion yếu
Không phát hiện.

Rủi ro maintainability

- Phụ thuộc vào trạng thái nội bộ của đối tượng: Các test TC-011 và TC-013 thay đổi trạng thái isActive của đối tượng target/account bằng cách gọi deactivate(). Nếu trong tương lai logic deactivate() thay đổi (ví dụ: yêu cầu balance = 0), các test này có thể trở nên flaky.
- Cách giảm rủi ro: Tạo một helper method để tạo tài khoản đã ở trạng thái inactive (ví dụ createInactiveAccount(String id, double balance)) thay vì gọi deactivate() trong test.
- Hard-coded giá trị delta cho double: Các assert assertEquals(expected, actual, 0.0001) sử dụng delta cố định; nếu thuật toán thay đổi cách tính (ví dụ: làm tròn), các test có thể thất bại không phải do lỗi logic.
- Cách giảm rủi ro: Sử dụng assertEquals(expected, actual, () -> "Message") hoặc assertEquals(expected, actual, Math.ulp(expected)*10) để linh hoạt hơn.

Gợi ý sửa ngay

1. Thêm test @TempDir

@TempDir

static Path sharedTempDir;

@Test

```
void testTempDirSharedFirstUse() throws IOException {  
    Path file = Files.createFile(sharedTempDir.resolve("a.txt"));  
    assertTrue(Files.exists(file));  
}
```

@Test

```
void testTempDirSharedSecondUse() {  
    assertTrue(Files.isDirectory(sharedTempDir));  
}
```

2. Thêm test migration annotation

// lớp mẫu JUnit4

@RunWith(MockitoJUnitRunner.class)

@Category(IntegrationTest.class)

public class OldTest { }

// test JUnit5 kiểm tra sự thay thế

@Test

```
void testMigrationAnnotations() {  
    Class<?> cls = OldTest.class;  
    assertTrue(cls.isAnnotationPresent(ExtendWith.class));  
    assertTrue(cls.isAnnotationPresent(Tag.class));  
}
```

```
}
```

3. Thêm test @ParameterizedTest argument conversion

```
static class Currency {  
    private final String code;  
    private Currency(String code) { this.code = code; }  
    public static Currency of(String s) { return new Currency(s); }  
    @Override public boolean equals(Object o) { return o instanceof Currency &&  
        ((Currency)o).code.equals(code); }  
}
```

```
@ParameterizedTest  
@ValueSource(strings = {"USD", "EUR"})  
void testCurrencyConversion(Currency cur) {  
    assertNotNull(cur);  
}
```

4. Tách helper tạo tài khoản inactive để giảm phụ thuộc trạng thái.

Áp dụng các sửa đổi trên sẽ hoàn thiện đầy đủ các yêu cầu kiểm thử đã nêu và giảm thiểu rủi ro bảo trì trong tương lai.